

LATIHAN SOAL

Pohon Biner

KAMUS UMUM

```
{ deklarasi dan inisiasi konstanta global }  
constant NIL = NULL  
  
{ deklarasi Abstract Data Type }  
type boolean: (true, false)  
type ElType: < key: integer, count: integer >  
type BinTree : pointer to TreeNode  
type TreeNode : < info : ElType,  
                    left : BinTree,  
                    right : BinTree >  
  
{ deklarasi fungsi dan prosedur pembantu, asumsikan saja seluruh  
  fungsi dan prosedur sudah tersedia }  
function newTreeNode(x: ElType) → BinTree  
procedure insSearchTree(input x: ElType, input/output p: BinTree)  
procedure delBTree(input/output p: BinTree, input x: ElType)  
function isEmpty(p: BinTree) → boolean  
procedure CreateTree(input x: ElType, input l: BinTree, input r:  
                    BinTree, output p: BinTree)  
procedure deallocTreeNode(p: BinTree)
```

IMPLEMENTASI FUNGSI

```
function bSearch (p: BinTree, x: ElType) → boolean  
{ Mengirimkan true jika ada node dari p (BST) yang bernilai x }
```

KAMUS LOKAL

```
{ Tidak ada kamus lokal }
```

ALGORITMA

```
if (p = NIL) then  
    → false  
else  
    depends on (p):  
        (p↑.info = x): → true  
        (p↑.info > x): → bSearch(p↑.left, x)  
        (p↑.info < x): → bSearch(p↑.right, x)
```

function isBTree(p: BinTree) → boolean

{ Mengirimkan true jika p adalah balanced tree. Asumsi: p tidak kosong }

KAMUS LOKAL

{ deklarasi fungsi helper }

function height(p: BinTree) → integer

function nodeCount(p: BinTree) → integer

hl, hr: integer

nl, nr: integer

diff_h, diff_n: integer

ALGORITMA

if (p = NIL) **then**

→ true

else

hl ← height(p↑.left)

hr ← height(p↑.right)

nl ← nodeCount(p↑.left)

nr ← nodeCount(p↑.right)

if (hl ≥ hr) **then**

diff_h ← hl - hr

else

diff_h ← hr - hl

if (nl ≥ nr) **then**

diff_n ← nl - nr

else

diff_n ← nr - nl

→ (diff_h ≤ 1) **and** (diff_n ≤ 1) **and** isBTree(p↑.left) **and**
isBTree(p↑.right)

[i] Implementasi fungsi height untuk menghitung tinggi dari *Binary Search Tree*

function height(p: BinTree) → integer

{ Menghitung tinggi dari *Binary Search Tree* }

KAMUS LOKAL

hl, hr: integer

ALGORITMA

```
if (p = NIL) then  
    → 0  
else  
    hl ← height(p↑.left)  
    hr ← height(p↑.right)  
    if (hl > hr) then  
        → hl + 1  
    else  
        → hr + 1
```

[1] Implementasi fungsi nodeCount untuk menghitung banyaknya node pada Binary Search Tree

```
function nodeCount(p: BinTree) → integer  
{ Menghitung banyaknya node pada Binary Search Tree }
```

KAMUS LOKAL

{ Tidak ada kamus lokal }

ALGORITMA

```
if (p = NIL) then  
    → 0  
else  
    → 1 + nodeCount(p↑.left) + nodeCount(p↑.right)
```

```
function buildBST(infos: array [0..99] of integer, nEff: integer) → BinTree  
{ Mengirimkan BST yang semua elemennya ada di infos (tidak terurut dan tidak unik). Infos selalu diisi dari 1. nEff adalah jumlah elemen infos yang terdefinisi, 0 jika kosong }
```

KAMUS LOKAL

i: integer
tempEl: ElType
root: BinTree

ALGORITMA

```
root ← NIL  
if (nEff > 0) then  
    i traversal [0..nEff-1]  
        tempEl.key ← infos[i]  
        tempEl.count ← 1  
        insSearchTree(tempEl, root)  
→ root
```